Transformer와 BERT

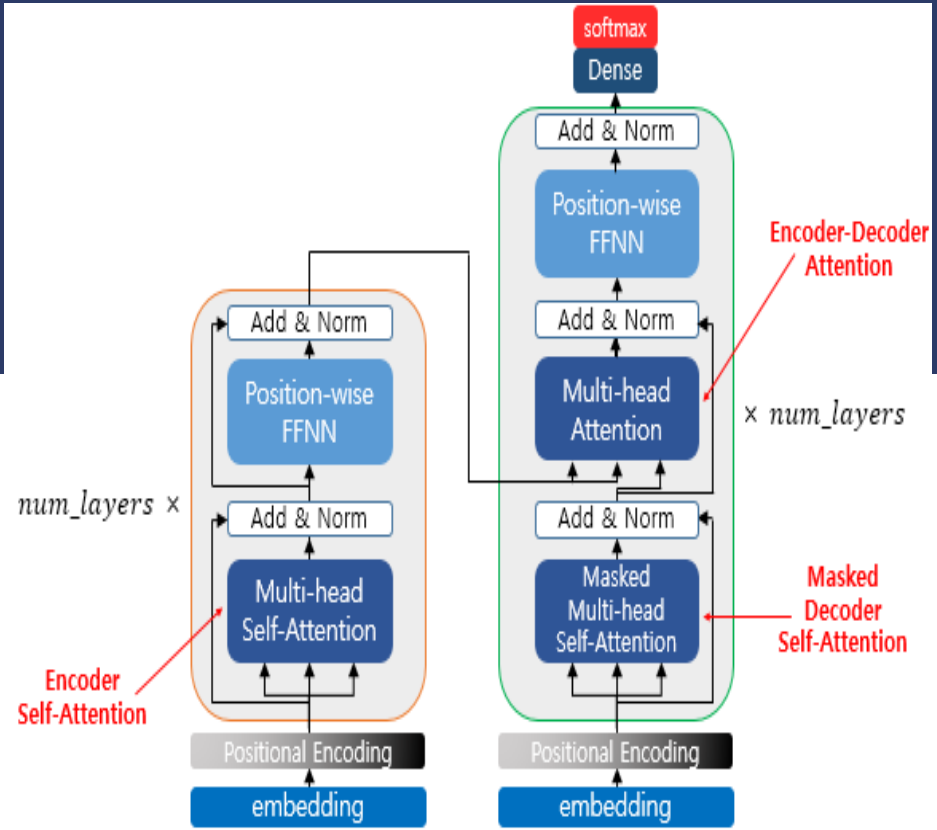
Attention Is All You Need

자연어 처리의 패러다임을 바꾼 구조

이종혁 (경희대학교 미디어학과)

Transformer

인코더-디코더 구조 · Attention · Self-Attention · Multi-Head Attention



왜 Transformer가 필요한가? — RNN의 한계 극복

RNN/LSTM은 시퀀스를 순차적으로 처리 → 병렬 처리 불가능(느림), 긴 문장에서 정보 손실.
Seq2Seq의 Context Vector 병목 문제: 아무리 긴 문장도 하나의 벡터로 압축 → 정보 소실.
2017년 Google "Attention Is All You Need" 논문: RNN 없이 Attention만으로 시퀀스 처리 → Transformer 탄생.

1 RNN의 문제점

순차 처리: 단어를 하나씩 순서대로 처리 → 병렬 처리 불가 → GPU 활용 제한

장기 의존성: 문장이 길면 초기 단어 정보가 뒤로 갈수록 사라짐 (LSTM으로도 완전 해결 안 됨)

Context Vector 병목: Seq2Seq에서 긴 문장을 하나의 벡터로 압축 → 손실

2 Transformer의 해결

Self-Attention: 모든 단어 간 관계를 한 번에 계산 → 병렬 처리 가능

거리 무관: 문장 내 아무리 멀리 떨어진 단어도 직접 관계를 파악

인코더-디코더 유지: Seq2Seq 구조는 유지하되, RNN을 Attention으로 대체

큰 그림 — 인코더와 디코더

Transformer는 번역기라고 생각하면 됩니다

인코더 (Encoder) — "읽는 뇌"

입력 문장을 깊이 이해하는 역할

- 입력 시퀀스를 문맥화된 표현으로 변환
- 같은 구조를 6층 반복
- 전체 입력을 동시에 처리 (병렬)

디코더 (Decoder) — "쓰는 뇌"

이해한 내용으로 출력을 생성하는 역할

- 인코더 출력을 참조하며 시퀀스 생성
- 한 단어씩 순서대로 출력
- 미래 단어는 볼 수 없음 (마스킹)

비유: 인코더는 통역사의 듣기 모드(원문을 완벽히 이해), 디코더는 말하기 모드(이해한 내용을 한 단어씩 전달)

입력 임베딩 + 위치 인코딩

임베딩 (Embedding)

각 단어(토큰)를 512차원의 숫자 벡터로 변환
"나는" → [0.12, -0.34, 0.56, ...] (512개 숫자)

위치 인코딩 (Positional Encoding)

"나는 학생입니다"에서 순서 정보가 사라지는 문제 해결.
각 위치에 고유한 신호를 더해줍니다.
→ 모델이 "나는"이 첫 번째, "학생입니다"가 두 번째라는 것을 알 수 있음.

주요 크기 | 임베딩 차원 (d_model): 512 | 어텐션 헤드: 8개 | 인코더/디코더 층: 각 6개

어텐션 (Attention) 메커니즘 — Q, K, V

핵심 아이디어: "출력을 생성할 때, 입력의 어디에 주목해야 하는가?"를 학습으로 결정하는 메커니즘

Query (Q) — 질문

"나는 누구와 관련이 있을까?"

카페 손님이 "아이스 아메리카노 있어요?"라고 질문하는 것.

입력 벡터 × W_Q로 생성

Key (K) — 열쇠

"나는 이런 정보를 갖고 있어"

카페 메뉴판의 각 항목이 자기 소개하는 것.

입력 벡터 × W_K로 생성

Value (V) — 실제 값

"내 실제 정보는 이거야"

메뉴 항목의 실제 내용물(커피 자체). Q와 K가 매칭되면 전달됨.

입력 벡터 × W_V로 생성

중요: W_Q, W_K, W_V는 모두 학습으로 결정. 모델이 훈련하면서 "어떤 관계를 봐야 잘 하는지" 스스로 학습.

Attention 계산 4단계

1

Q와 K의 내적 → 유사도 점수

"학생입니다"의 Q와 모든 단어의 K를 내적.
값이 클수록 관련이 깊음 → "이 단어가 다른 단어와 얼마나 관련 있는가?"

2

√d_k로 나누기 → 스케일링

d_k=64일 때 √64=8로 나눔.
값이 너무 크면 Softmax가 한 곳에만 몰림 → 적당한 크기로 조절

3

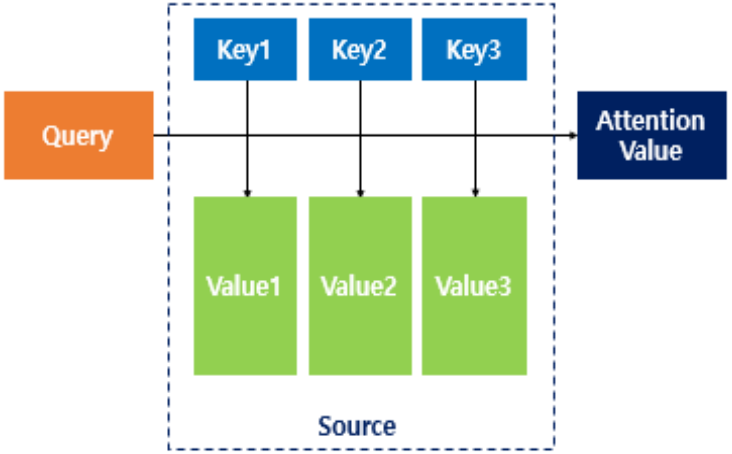
Softmax → 확률 분포로 변환

모든 점수의 합이 1이 되는 확률로 변환.
"나는"=0.3, "학생입니다"=0.7 → 어디에 얼마나 주목할지 결정

4

가중치로 V를 곱해서 합산

0.3×V("나는") + 0.7×V("학생입니다")
→ 관련 정보가 반영된 새로운 벡터 탄생!




```
Attention(Q,K,V) = Softmax(Q·K^T / √d_k) × V
```

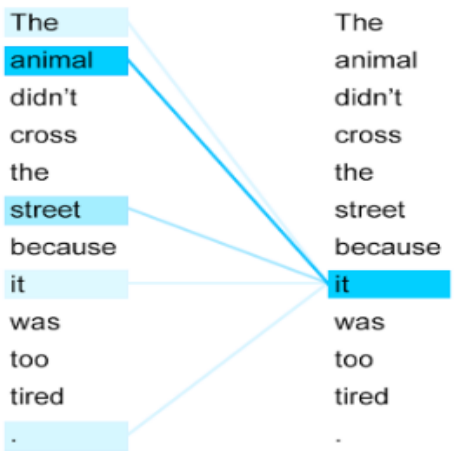
셀프 어텐션 (Self-Attention) — 문장 내부의 관계

일반 Attention (Encoder-Decoder)

Q = 디코더에서, K·V = 인코더에서
→ "번역할 때 원문의 어디를 볼까?"
→ 두 개의 다른 시퀀스 간의 관계

Self-Attention

Q, K, V 모두 같은 시퀀스에서
→ "이 문장 내에서 단어 간 관계는?"
→ 한 문장 안의 모든 관계를 스스로 파악



거리 무관

아무리 멀리 떨어진
단어도 직접 연결

병렬 처리

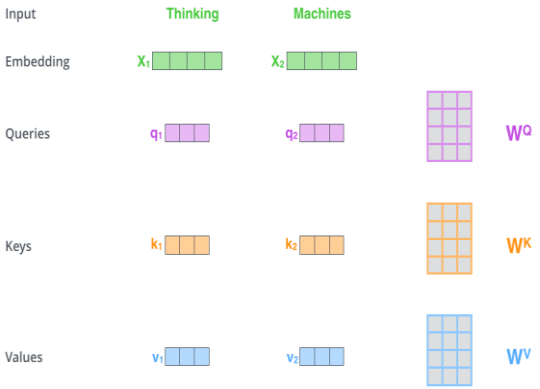
모든 단어 쌍의 관계를
동시에 계산 (GPU 활용)

해석 가능성

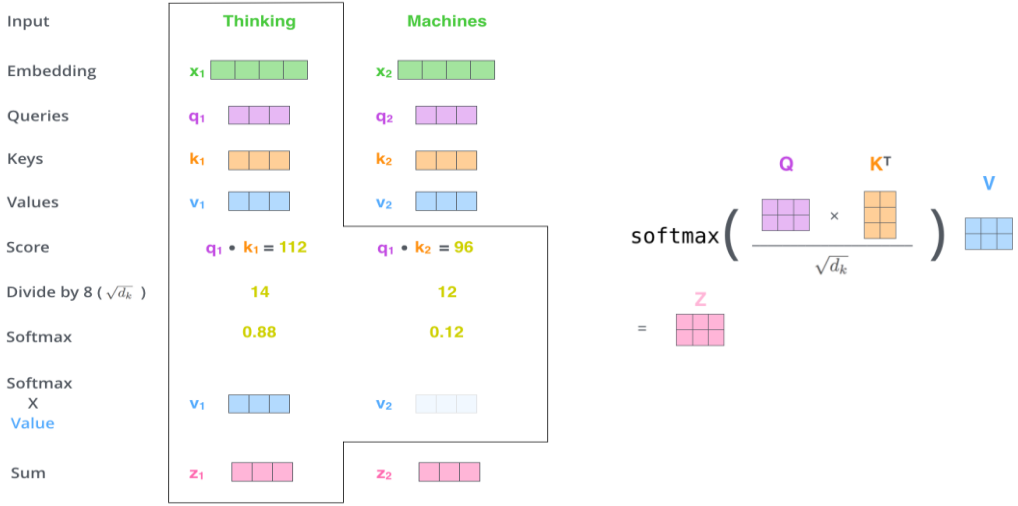
가중치 시각화로
모델의 주목점 확인

<부록> 셀프 어텐션 (Self-Attention) – 계산

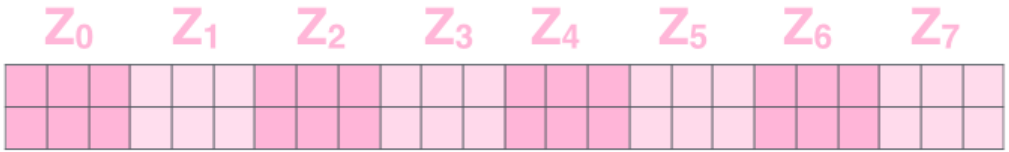
#1
-한 문장을 구성하는 단어(토큰)별로 embedding에서 Q,K,V 행렬 만들기
-Q, K, V weights는 업데이트될 파라미터



#2
-각 단어별로 모든 단어들에 대해 Q,K 내적 값 산출
-이를 multi-head 수(8)로 나눠 softmax 산출.
-이를 가중치로 V에 곱한 뒤 모든 단어의 결과 벡터를 더하면 해당 단어의 attention score 도출
-이 수치는 단어 자신과 다른 모든 단어와의 관계를 반영



#3
-Multi-head 수만큼 나타난 attention score 벡터들을 붙여 본래 input embedding 차원의 벡터로 만듦.



멀티 헤드 어텐션 (Multi-Head Attention)

왜 여러 개의 헤드가 필요한가? 하나의 헤드는 하나의 관점만 포착. 언어에는 여러 종류의 관계가 동시에 존재.

예: "철수가 어제 학교에서 영희에게 책을 줬다"

헤드1: 주어-동사 (철수-줬다) | 헤드2: 의미적 (영희-줬다) | 헤드3: 위치 (학교-줬다) | 헤드4: 시간 (어제-줬다)

① 분할 (Split)

Q, K, V를 8개 헤드로 분할
512차원 ÷ 8 = 각 64차원

각 헤드가 서로 다른
부분 공간(subspace)에서
독립적으로 Attention 계산

② 병렬 Attention

8개 헤드가 동시에
각자의 Attention 수행

각 헤드는 다른 패턴 학습:
문법적, 의미적, 위치적
관계 등을 각각 담당

③ 합치기 (Concat+Linear)

8개 출력(각 64차원)을
이어붙여 → 512차원

선형 변환(Projection)을
한 번 더 거쳐 최종
Multi-Head Attention 출력

```
MultiHead(Q,K,V) = Concat(head_1, …, head_8) × W_O    |    head_i = Attention(Q·W_Qi, K·W_Ki, V·W_Vi)
```

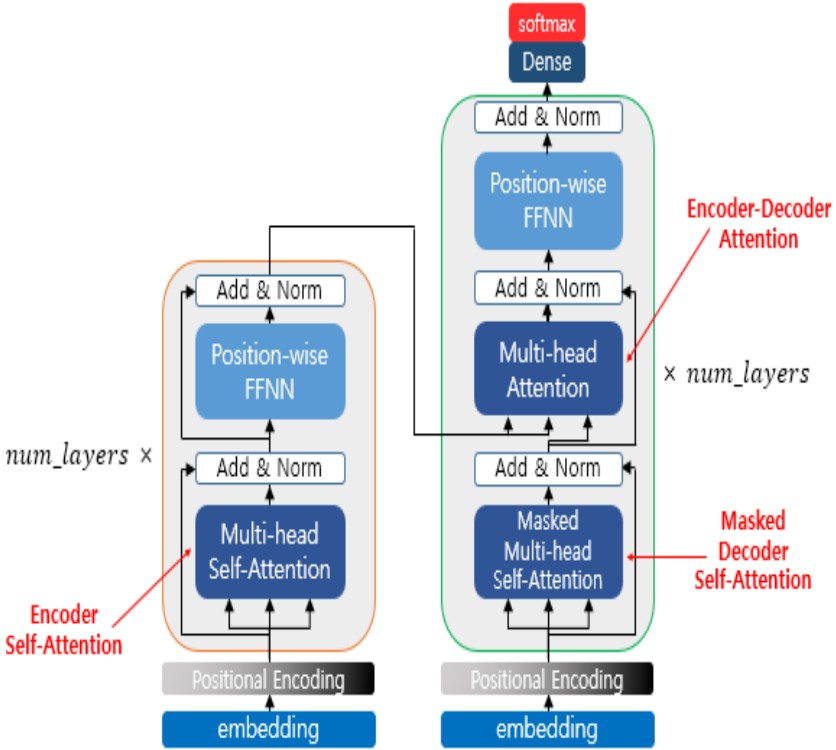
인코더 내부 구조 — 6층 반복

1 **Multi-Head Self-Attention**
모든 단어가 서로를 바라보며 관계 파악.
8개 헤드가 각각 다른 관점으로 분석.

2 **Add & Norm (잔차 연결 + 정규화)**
Add: 원래 입력을 다시 더해 정보 손실 방지.
Norm: 숫자 스케일을 일정하게 맞춰 안정화.

3 **Position-wise FFNN**
각 단어의 표현을 개별적으로 풍부하게.
512→2048→512 확장/축소, 비선형 패턴 학습.

4 **Add & Norm**
FFNN 결과에도 잔차 연결 + 정규화 적용.
매 단계마다 반복하여 학습 안정성 확보.



1층: 단어 수준의 관계 파악 → 6층: 문장 전체의 의미와 뉘앙스까지 이해 (점점 깊은 이해로)

디코더 내부 구조 — 6층 반복

1 **출력 임베딩 + 위치 인코딩**
지금까지 생성한 출력 단어들을 벡터로 변환

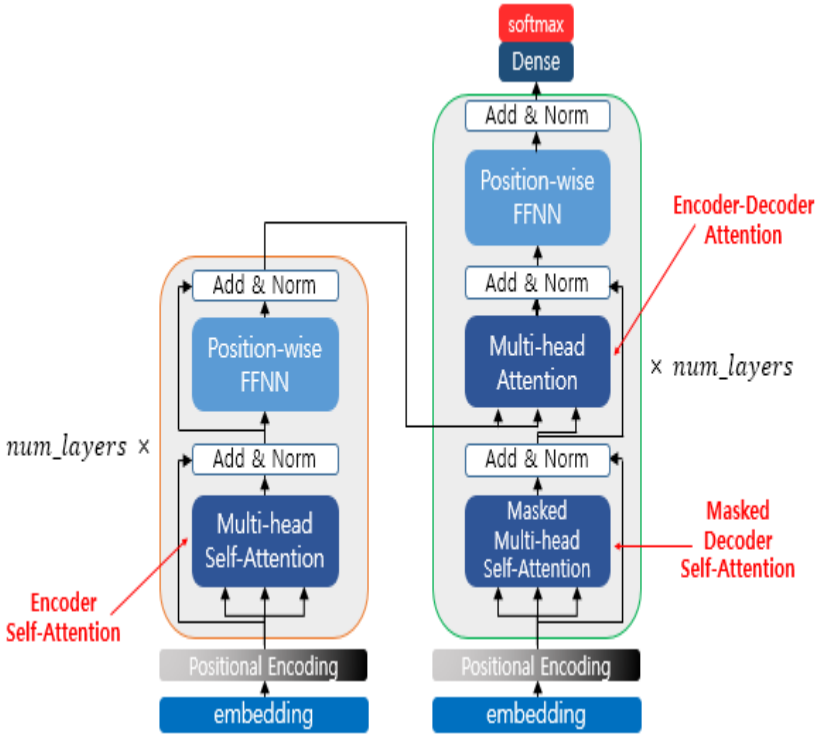
2 **Masked Multi-Head Self-Attention**
미래 단어를 못 보게 마스킹 (시험에서 뒷문제 답 못 보는 것)

3 **Add & Norm**
잔차 연결 + 정규화

4 **Encoder-Decoder Attention (cross-attention)**
Q=디코더, K·V=인코더 출력. "원문 어디를 참고할까?" 핵심!

5 **Add & Norm**
잔차 연결 + 정규화

6 **Position-wise FFNN + Add & Norm**
각 단어 표현을 풍부하게 + 안정화



디코더의 핵심 차이: Masked Attention(미래를 가림) + Encoder-Decoder Attention(인코더에게 물어봄)

전체 흐름 – 번역 예시로 보기

입력: "나는, 학생입니다" → 출력: "I am a student"

한국어 입력 → 임베딩 +위치 → 인코더 ×6 → 문맥 벡터 → 디코더 ×6 → Dense +Softmax → 영어 출력

1. 각 한국어 단어 → 임베딩 + 위치 인코딩으로 512차원 벡터 변환
2. 인코더 6층 통과 → 각 단어가 문맥을 풍부하게 담은 벡터로 변환됨
3. 디코더에 <sos> 입력 → 인코더 출력 참조 → "I" 생성
4. 디코더에 <sos>, I 입력 → "am" 생성 → 반복 → "a" → "student" → <eos>

핵심: 인코더는 입력 전체를 동시에 이해하고, 디코더는 하나씩 생성하며 필요할 때 인코더에게 물어본다

BERT

Bidirectional Encoder Representations from Transformers

2018년 Google이 공개. Transformer의 인코더를 여러 개 쌓아 올린 구조.

위키피디아(25억 단어) + BooksCorpus(8억 단어)로 사전 훈련 → 특정 과제에 Fine-tuning하면 NLP 최고 성능.

핵심: 양방향(Bidirectional) 문맥 이해 — 좌→우, 우→좌 양쪽 문맥을 동시에 활용.

Transformer에서 BERT와 GPT로

1 Transformer (2017)

인코더 + 디코더
번역 등 시퀀스 변환 과제
"입력 → 출력" 구조

→

2 BERT (2018)

인코더만 사용
양방향으로 문맥 이해
"입력을 이해" 하는 모델

→

3 GPT (2018~)

디코더만 사용
단방향(좌→우) 생성
"텍스트를 생성" 하는 모델

BERT 구조 — 인코더 스택

1 BERT-Base

L = 12 (인코더 층 수)

D = 768 (임베딩 차원, d_model)

A = 12 (셀프 어텐션 헤드 수)

파라미터 수: 1억 1천만 개 (110M)

초기 Transformer(L=6, D=512, A=8)보다 큰 규모

2 BERT-Large

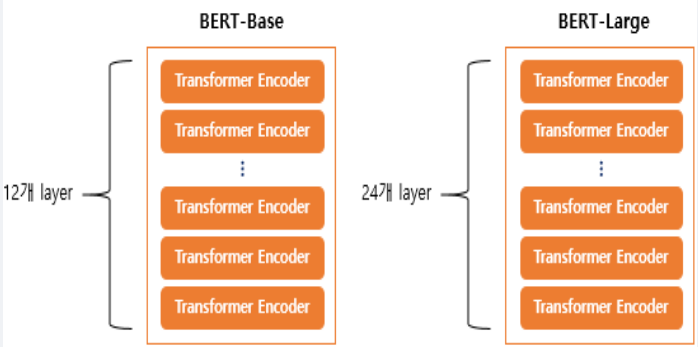
L = 24 (인코더 층 수)

D = 1024 (임베딩 차원)

A = 16 (셀프 어텐션 헤드 수)

파라미터 수: 3억 4천만 개 (340M)

BERT-Base보다 3배 큰 규모



3 입력 구성

최대 시퀀스 길이: 512 토큰

두 문장까지 입력 가능

특수 토큰: [CLS] = 문장 시작 (분류용), [SEP] = 문장 구분/끝

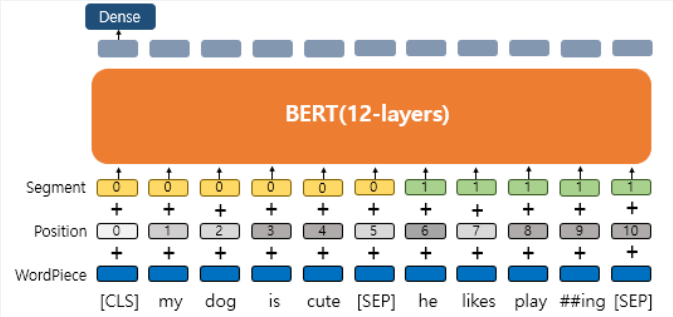
3가지 임베딩의 합: 토큰 임베딩 + 세그먼트 임베딩 + 위치 임베딩

4 WordPiece 토크나이징

단어보다 작은 단위(서브워드)로 분리

자주 등장하는 단어: 그대로 유지. "here", the"

드문 단어: 서브워드로 분리. "embeddings" → "em", "##bed", "##ding", "##s“ → OOV(미등록 단어) 문제 해결

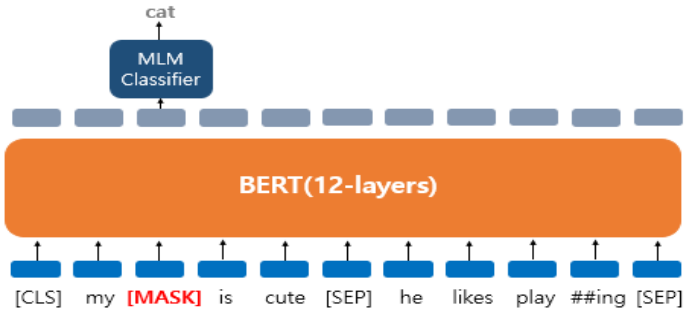


BERT 사전학습 (1) — 마스크드 언어 모델 (MLM)

Masked Language Model (MLM)

입력 단어의 15%를 무작위로 선택하여 가려진 단어를 예측하도록 학습

> 예시: "나는 [MASK]에 가서 국어와 [MASK]를 공부했다"
> → 모델이 "학교"와 "수학"을 맞추도록 학습



1 **80% → [MASK]로 변경**
실제로 마스킹 처리

"나는 [MASK]에 가서..."

모델이 빈칸을 채우는
핵심 학습 방식

2 **10% → 랜덤 단어로 변경**
다른 단어로 교체

"나는 바나나에 가서..."

모델이 잘못된 단어를 감지하고
올바른 단어를 추론하도록 학습

3 **10% → 그대로 유지**
원래 단어 유지

"나는 학교에 가서..."

[MASK] 토큰에만 의존하지 않고
실제 문맥에서도 학습하도록 유도

BERT 사전학습 (2) — 다음 문장 예측 (NSP)

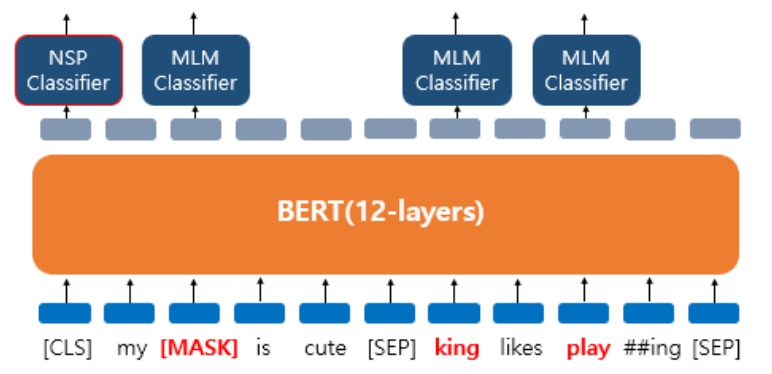
Next Sentence Prediction (NSP)

두 개의 문장을 주고 두 번째 문장이 이어지는 문장인지 맞추는 학습.

실제 이어지는 문장 쌍 50% + 무작위로 이어붙인 문장 쌍 50%로 학습.

이 학습을 통해 문장 간 관계를 이해하는 능력을 갖추게 됨

→ QA, 자연어 추론 등에 활용



1 IsNext (실제 연속 문장)

문장 A: "The man went to the store."

문장 B: "He bought a gallon of milk."

→ Label: IsNext (연속)

두 번째 문장이 첫 번째 문장 뒤에
자연스럽게 이어지는 경우

2 NotNext (무작위 문장)

문장 A: "The man went to the store."

문장 B: "Penguins are flightless birds."

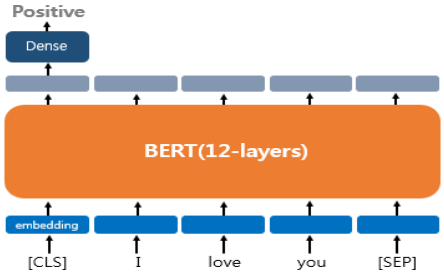
→ Label: NotNext (불연속)

완전히 무관한 문장을
무작위로 이어붙인 경우

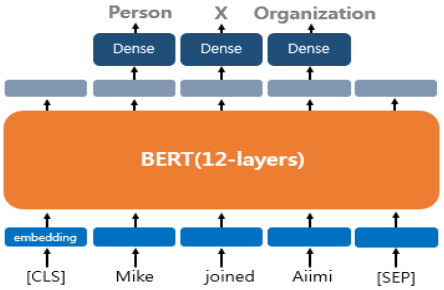
BERT 파인 튜닝 (Fine-tuning) — 전이학습

전이학습(Transfer Learning): 사전학습(Pre-training)된 BERT를 가져와, 맞춤형 데이터로 간단한 파인 튜닝(파라미터 재조정)만으로 학습

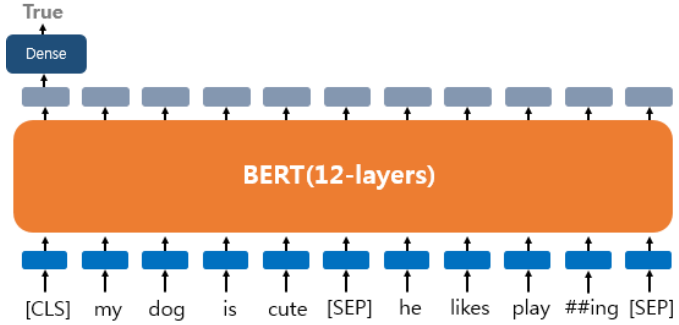
1 한 문장의 성격 분류



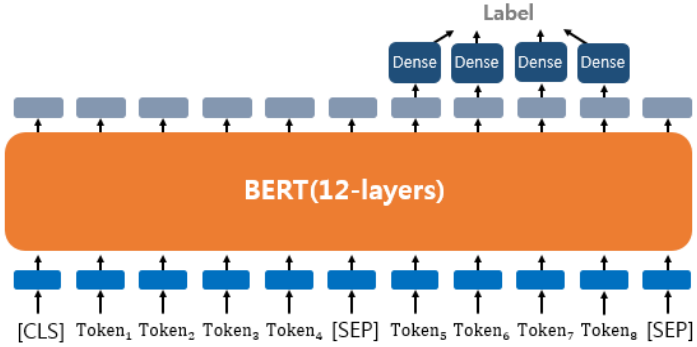
2 단어에 대한 태깅: 개체명이나 품사 분석



3 문장 간 관계 분석: 자연어 추론



4 새로운 내용 생성: 질의 응답



BERT vs Sentence-BERT (SBERT)

BERT: "이 단어가 이 문맥에서 무슨 뜻이지?" — 단어 수준 이해 | **SBERT:** "이 문장과 저 문장이 얼마나 비슷하지?" — 문장 수준 비교

BERT — 단어 수준 이해

출력: 각 단어마다 벡터 1개 (768차원)
→ 문장 전체를 대표하는 벡터가 없음

비유: 매번 두 사람을 나란히 세워서 비교

비교 방식: 두 문장을 반드시 함께 입력
[CLS] 문장A [SEP] 문장B [SEP]

1만 건 비교: 수 시간 소요
(5천만 번 추론 필요)

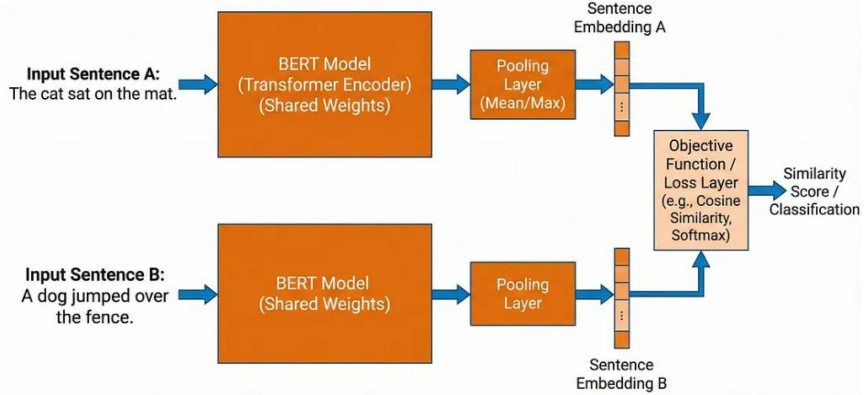
SBERT — 문장 수준 비교

출력: 문장 전체 → 벡터 1개 (지문/fingerprint)
→ 풀링(Pooling)으로 단어벡터를 요약

비유: 각자 여권사진을 미리 찍어두고 비교

비교 방식: 각 문장을 독립적으로 인코딩 후
코사인 유사도로 즉시 비교

1만 건 비교: 수 초 완료
(벡터 연산만으로 끝)



허깅페이스 BERT 모델

BERT 계열 (transformers)

`nlptown/bert-base-multilingual-uncased-sentiment`
다국어 감성분석 (1~5점)

`beomi/KcELECTRA-base-v2022`
한국어 파인튜닝 범용 기반 모델

`klue/roberta-large`
한국어 NLU 벤치마크

`dslim/bert-base-NER`
영어 개체명 인식

`monologg/koelectra-base-v3-finetuned-korquad`
한국어 QA

SBERT 계열 (sentence-transformers)

`jhgan/ko-sroberta-multitask`
한국어 SBERT (추천)

`snunlp/KR-SBERT-V40K-klueNLI-augSTS`
한국어 SBERT (KLUE 학습)

`all-MiniLM-L6-v2`
영어 (빠르고 가벼움, 384차원)

`all-mpnet-base-v2`
영어 고성능 (768차원)

`intfloat/multilingual-e5-large`
다국어 (한국어 포함, 1024차원)

허깅페이스 BERT 모델 - 추론 코드(사례)

BERT 계열 (transformers)

```
from transformers import pipeline

classifier = pipeline("sentiment-analysis",
    model="nlptown/bert-base-multilingual-
           uncased-sentiment")

print(classifier("이 영화 정말 재미있었어요!"))
# [{'label': '5 stars', 'score': 0.72}]
```

SBERT 계열 (sentence-transformers)

```
from sentence_transformers import SentenceTransformer, util

model = SentenceTransformer("jhgan/ko-sroberta-multitask")

emb = model.encode([
   "한국 경제가 1분기에 마이너스 성장을 기록했다.",
   "GDP 성장률이 음수를 나타냈다.",
   "오늘 날씨가 매우 좋습니다.",
])

print(util.cos_sim(emb[0], emb[1]).item()) # 0.89 (높음)
print(util.cos_sim(emb[0], emb[2]).item()) # 0.12 (낮음)
```

Transformer 기반 모델의 활용 분야

1 자연어 처리 (NLP)

기계 번역: 다국어 자동 번역

감정 분석: 텍스트 감성 판별

질의 응답: 문맥에서 답변 추출

텍스트 요약: 긴 문서 자동 요약

개체명 인식: 인물/장소/기관 추출

2 컴퓨터 비전 (Vision)

ViT: 이미지 분류 (Part 2 참조)

DETR: 객체 탐지

SegFormer: 이미지 세분화

Swin Transformer: 계층적 비전

→ CNN의 영역을 빠르게 침투

3 미디어/저널리즘 연구

BERT로 뉴스 감정 분석

가짜 뉴스 탐지

뉴스 기사 자동 분류

정보원 추출 및 분석

프레이밍 분석 자동화

4 멀티모달 AI

텍스트 + 이미지 통합 처리

CLIP: 텍스트-이미지 연결

DALL-E: 텍스트에서 이미지 생성

Flamingo: 시각-언어 모델

→ Transformer가 핵심 구조

핵심 용어 정리

용어	설명	비유
Self-Attention	한 문장 내 모든 단어 쌍의 관련성 계산	학생이 자기 노트의 모든 내용을 상호 참조
Multi-Head	여러 관점에서 동시에 어텐션 계산	8명이 같은 글을 각자 다른 관점에서 분석
Q, K, V	질문(Query), 열쇠(Key), 값(Value)	도서관에서 질문으로 책(Key)을 찾아 내용(V) 확인
Positional Encoding	단어의 순서 정보를 임베딩에 추가	책의 페이지 번호를 내용에 표시
Add & Norm	잔차 연결 + 레이어 정규화	새 내용을 배울 때 기존 지식도 함께 유지
Masked Attention	미래 단어를 보지 못하게 마스킹	시험 중 다음 문제 미리보기 방지
MLM	빈칸 채우기 학습 (BERT)	빈칸 뚫린 문장 완성하기 시험
NSP	다음 문장 예측 학습 (BERT)	두 문장이 이어지는지 판단하는 독해 문제
Fine-tuning	사전학습 모델을 특정 과제에 맞게 조정	일반 교육 받은 학생이 전공 과목 심화 학습